

Node.js Modules – os, path and fs

Student Name: P. Dharani	Roll No.: A24126552261
Course: Full Stack Web Development	Course Code: 23CM4121
Experiment No.: 6-6	

1 Title

Node.js Modules — os, path and fs

2 Aim

To understand Node.js modules and demonstrate the use of the built-in `os`, `path` and `fs` modules for operating-system information, file-path handling and file operations.

3 Objective

- To understand modularity in Node.js.
- To use the built-in `os` module to obtain system information.
- To use the `path` module to construct and manage file paths.
- To use the `fs` module to create, write and read a file.
- To display and verify the results through the Node.js terminal.

4 Theory

Node.js modules are reusable pieces of functionality that can be imported into a Node.js application. Node.js provides several built-in modules that can be used without installing external packages.

Module	Purpose and methods used in this experiment
os	Gives information about the operating system and hardware: <code>os.platform()</code> , <code>os.arch()</code> , <code>os.hostname()</code> and <code>os.cpus()</code> .
path	Utilities for working with file and directory paths. <code>path.join()</code> builds a valid path without manually typing path separators.
fs	File-system operations. <code>fs.writeFileSync()</code> writes data to a file and <code>fs.readFileSync()</code> reads data from a file.

Basic flow: Node.js Application → [`os`: System Information | `path`: File Path | `fs`: File Creation and Reading] → Terminal Output

5 Algorithm / Procedure

1. Create a Node.js project folder.
2. Create an `app.js` file.
3. Import the built-in `os`, `path` and `fs` modules.
4. Obtain operating-system information using the `os` module.
5. Create the path of `student.txt` using the `path` module.
6. Write student information into the file using the `fs` module.
7. Read the file using `fs.readFileSync()`.
8. Display the system information, path information and file contents in the terminal.
9. Verify that the file was created successfully and that its contents were read correctly.

6 Program

Project structure

```
Q6-NodeJS-Modules/
├── src/
│   ├── app.js
│   └── student.txt    (created when app.js runs)
```

app.js

```
// Experiment 6-6: Node.js built-in modules (os, path, fs)

// Import the built-in modules
const os = require("os");
const path = require("path");
const fs = require("fs");

// ----- 1. os module -----
function showSystemInfo() {
  console.log("----- SYSTEM INFORMATION (os) -----");
  console.log("Platform      :", os.platform());
  console.log("Architecture  :", os.arch());
  console.log("Hostname      :", os.hostname());
  console.log("Number of CPUs  :", os.cpus().length);
}

// ----- 2. path module -----
function showPathInfo(fullPath) {
  console.log("\n----- PATH INFORMATION (path) -----");
  console.log("Full path      :", fullPath);
  console.log("File name     :", path.basename(fullPath));
  console.log("Extension     :", path.extname(fullPath));
  console.log("Folder        :", path.dirname(fullPath));
}

// ----- 3. fs module -----
function createAndReadFile(fullPath) {
  const lines = [
    "Student Name: P. Dharani",
    "Course: Full Stack Web",
    "Development", "Course Code:",
    "23CM4121",
  ];
  const topic = "Topic: Node.js Modules";

  // write the file (creates it, or overwrites it if it already exists)
  fs.writeFileSync(fullPath, lines.join("\n"));
  console.log("\n----- FILE OPERATIONS (fs) -----");
  console.log("File created successfully.");
  console.log("File exists    :", fs.existsSync(fullPath));

  // read the same file back
  const content = fs.readFileSync(fullPath, "utf8");
  console.log("\nContents of student.txt:");
  console.log(content);
}

// ----- main program -----
const studentFile = path.join(__dirname, "student.txt");
showSystemInfo();
showPathInfo(studentFile);
createAndReadFile(studentFile);
```

7 Code Explanation

Code / Method	Explanation
<code>require("os"), require("path"), require("fs")</code>	Import the three built-in modules. They ship with Node.js, so no installation is needed.
<code>showSystemInfo()</code>	Function that prints the system details using the <code>os</code> module.
<code>os.platform(), os.arch()</code>	Return the operating-system platform (e.g. linux, win32) and the processor architecture (e.g. x64).
<code>os.hostname()</code>	Returns the name of the computer.
<code>os.cpus().length</code>	<code>os.cpus()</code> returns an array with one entry per CPU core, so its length is the number of CPUs.
<code>path.join(__dirname, "student.txt")</code>	Joins the folder of the running script (<code>__dirname</code>) and the file name into one valid path, without typing separators by hand.
<code>path.basename(), path.extname(), path.dirname()</code>	Split the path into the file name (student.txt), the extension (.txt) and the folder.
<code>fs.writeFileSync(fullPath, lines.join("\n"))</code>	Joins the four text lines with line breaks and writes them to student.txt. The file is created, or overwritten if it already exists.
<code>fs.existsSync(fullPath)</code>	Returns true when the file is present, which confirms that the file was created.
<code>fs.readFileSync(fullPath, "utf8")</code>	Reads the whole file synchronously and returns its contents as text.
<code>console.log()</code>	Displays the results in the terminal.

8 Test Cases and Execution Data

The program was executed with Node.js and the terminal output was used for verification.

Test Case	Test / Input	Expected Result	Actual Result	Status
TC-01	Run node app.js	Platform, architecture, hostname and CPU count are displayed.	Platform: linux; architecture: x64; hostname: vm; CPUs: 1 (matches <code>os.cpus().length</code> checked separately).	Pass
TC-02	Path handling	Absolute path ending in student.txt; file name student.txt; extension .txt.	Full path /home/student/Q6-NodeJS-Modules/src/student.txt; file name student.txt; extension .txt.	Pass
TC-03	File creation	"File created successfully." is shown and <code>fs.existsSync()</code> is true.	Message displayed; File exists: true.	Pass
TC-04	Read the created file	The four lines written to the file are read back unchanged.	Student Name: D. Vijaya Sri; Course: Full Stack Web Development; Course Code: 23CM4121; Topic: Node.js Modules.	Pass
TC-05	Run the program a second time (special case)	No error; the file is overwritten, not appended, so it still has the same 4 lines.	Second run printed identical output; student.txt unchanged (same checksum, still 4 lines).	Pass

9 Output

Output 1: Node.js execution in the terminal

```
Terminal - student@vm: ~/Q6-NodeJS-Modules/src

student@vm:~/Q6-NodeJS-Modules/src$ node app.js
----- SYSTEM INFORMATION (os) -----
Platform      : linux
Architecture  : x64
Hostname      : vm
Number of CPUs : 1

----- PATH INFORMATION (path) -----
Full path     : /home/student/Q6-NodeJS-Modules/src/student.txt
File name     : student.txt
Extension     : .txt
Folder       : /home/student/Q6-NodeJS-Modules/src

----- FILE OPERATIONS (fs) -----
File created successfully.
File exists   : true

Contents of student.txt:
Student Name: P. Dharani
Course: Full Stack Web Development
Course Code: 23CM4121
Topic: Node.js Modules
student@vm:~/Q6-NodeJS-Modules/src$
```

Observed execution details

Item	Observed Value
Platform	linux
Architecture	x64
Hostname	vm
Number of CPUs	1
File path	/home/student/Q6-NodeJS-Modules/src/student.txt
File operation	File created successfully. (File exists: true)
File content	Student Name: D. Vijaya Sri; Course: Full Stack Web Development; Course Code: 23CM4121; Topic: Node.js Modules

Note: platform, hostname and CPU count depend on the computer on which the program is run, so they will differ on another machine.

10 Result

The `os`, `path` and `fs` built-in modules of Node.js were successfully demonstrated. The program retrieved operating-system information, generated a file path, created `student.txt` and read its contents back. The execution was verified using the actual terminal output.

End of Observation Record